

本地知识库构建方案

基于 RAG + GraphRAG 的混合检索架构 · 面向中文场景的本地化、私有化、可落地实现

离线构建 + 在线问答

向量库 + 知识图谱双通道

本地模型 · 数据不出域

技术栈 2025-2026

目录

- ① 思路正确性分析 ② 完善后的技术架构 ③ 分阶段实现路径 ④ 系统架构图 ⑤ 业务流程图
⑥ 技术选型清单 ⑦ 关键实现示例

1 原思路文件的正确性分析

参考文件：构建本地知识库流程思路.txt。整体判断：思路方向**正确且先进**，本质上是一套「RAG + 知识图谱 (GraphRAG) 混合检索 + 查询改写 + 重排」的完整范式，与业界 2024-2026 的主流最佳实践高度一致。

总体结论：核心链路（切分 → 向量化 → 图谱 → 查询改写 → 混合检索 → 重排 → 生成）设计合理、可落地。原方案已经涵盖了生产级 RAG 的绝大多数关键环节，不存在方向性错误。需要补齐的是「稀疏检索/混合检索」「最新模型选型」「评估与观测」「Agentic / 自纠错」等增强项，以及把模糊的组件具体化。

✓ 原思路中「做对了」的核心设计

- 按文件类型差异化解析与切分：**不同文档（PDF/Word/Markdown/扫描件）用不同策略，是正确的工程化思路，避免「一刀切」导致版面信息丢失。
- 用 MarkItDown / MinerU 做文档解析：**这是当前开源界最主流的「转 Markdown + 保留结构」方案，MinerU 对公式、表格、图表还原尤其强。
- 向量库 + 知识图谱双通道：**这正是 Microsoft GraphRAG / LightRAG 的核心思想——向量解决「语义相似」，图谱解决「实体关系与全局摘要」，互补能显著提升复杂问答（如「某公司近年战略演变」）的准确率。
- 查询改写(Rewrite)与扩充(Expansion)：**对应业界 Query Transformation，能显著改善口语化、含糊问题的召回，是性价比最高的增强手段之一。
- 图谱实体辅助扩充问题：**用图谱实体回填 query，是 GraphRAG 的标准做法，正确。

- 「相似度高 ≠ 相关」→ 引入 Rerank：用 Cross-Encoder 重排是正确且必要的，单纯向量相似度 top-k 噪声很大。
- Prompt 压缩：上下文过长时压缩（如 LLMingua）避免截断与高延迟，思路正确。

⚠ 原思路的缺失项 / 可增强点（需要「用最新技术完善」的地方）

原方案缺口	风险	最新技术补强
只提「向量相似度检索」	纯稠密检索对专名、编号、关键词召回差	混合检索（BM25 稀疏 + 向量稠密 + 图谱），用 Reciprocal Rank Fusion 融合
Embedding / Reranker 模型未指定	中文效果与性能不确定	BGE-M3 Qwen3-Embedding 做嵌入； BGE-Reranker Qwen3-Reranker 做重排
图谱「大模型分析构建」太笼统	抽取质量、成本、社区摘要不可控	采用 LightRAG / Microsoft GraphRAG 的标准化抽取 + 社区聚类 + 全局/本地检索模式
查询改写只列了 Rewrite/Expansion	对复杂/多跳问题不足	补充 HyDE、RAG-Fusion（多 Query）、Step-back、子查询分解
缺少评估环节	无法量化「变好没」、易线上翻车	RAGAS / DeepEval 做忠实度、相关性、上下文召回评估；建黄金测试集
缺少观测与缓存	排障难、成本高	Langfuse 观测链路、回答溯源；向量/答案缓存；限流鉴权
未考虑「检索不到怎么办」	幻觉或拒答体验差	Self-RAG / Corrective-RAG：低置信度时自纠错、澄清或转其他源
切分策略单一	长文档上下文断裂	语义切分、层级/父子切分、Late Chunking（保留全局上下文）

2 完善后的技术架构（2025–2026 最新）

在保留原方案「双通道 + 查询改写 + 重排」骨架基础上，补齐混合检索、最新模型、评估观测与自纠错，形成可生产的本地知识库系统。下面按分层给出具体技术与选型。

① 数据接入

② 文档解析

- 本地目录 / NAS 监听
- 对象存储 (S3/MinIO)
- 数据库导出、Web 抓取
- 增量同步 + 版本管理

- **Markdown**: 结构化文档→MD
- **MinerU**: PDF/公式/表格/图表
- **unstructured**: 多格式通用
- **PaddleOCR / PP-Structure**: 扫描件
- **统一输出带版面元数据的 JSON/MD**

③ 智能切分

- **语义切分** (按 embedding 距离断点)
- **层级/父子切分** (Small-to-Big)
- **Late Chunking**: 长上下文整体编码再切
- 按 Markdown 标题递归切分
- 重叠窗口, 保留章节/来源元数据

④A 向量索引

- Embedding: **BGE-M3 / Qwen3-Embedding**
- 稠密+稀疏+多向量一体
- 向量库: **Qdrant** / Chroma / LanceDB
- 支持元数据过滤、多租户

④B 图谱索引

- **LightRAG / Microsoft GraphRAG**
- LLM 抽取 实体-关系-社区
- 图存储: Neo4j 或复用向量库
- Local + Global 双模式检索
- 三元组文本化参与召回

⑤ 检索增强

- Query Transform: Rewrite/Expansion/HyDE/RAG-Fusion
- **混合检索**: BM25 + 向量 + 图谱
- Rerank: **BGE/Qwen3-Reranker**
- 压缩: **LLMLingua-2**
- Self-RAG 自纠错兜底

⑥ 生成与支撑

- 本地 LLM: **Ollama / vLLM**
- 模型: Qwen3 / DeepSeek / ChatGLM
- 回答带**引用溯源** (原文定位)
- 评估 **RAGAS** · 观测 **Langfuse**
- API(FastAPI) + 前端(Streamlit/Gradio)

关于「本地化 / 数据不出域」: 所有组件均可私有化部署——解析、Embedding、Reranker、LLM 跑在本地 GPU/CPU; 向量库与图谱库自托管。仅在你们确有合规要求允许时, 才考虑调用云端大模型 API 作为兜底。

为什么是「混合检索」而不是纯向量?

向量检索擅长语义，但对「合同编号 HT-2024-007」「错误码 0x80070005」这类专名/标识符召回很差。BM25 稀疏检索正好补强精确词面匹配；图谱补强实体关系与全局摘要；三者用 **Reciprocal Rank Fusion (RRF)** 融合后再交给 Reranker，是 2025 年公认的最优基线组合。

3 分阶段实现路径（里程碑）

Phase 0 · 环境与选型

约 3-5 天

- 清点语料规模、文档类型、硬件（GPU 显存 / 内存 / 磁盘）。
- 确定模型与存储：Embedding=BGE-M3、Reranker=BGE-Reranker-v2-m3、LLM=Qwen3（Ollama）、向量库=Qdrant、图谱=LightRAG。
- 用 Docker Compose 起 Qdrant / Neo4j / Langfuse；Python venv 隔离依赖。

Phase 1 · 文档接入与解析

约 1-2 周

- 实现多源连接器（本地目录监听 / MinIO / DB 导出）。
- 接 Markdown + MinerU + PaddleOCR，输出统一 {text, layout_meta, source, page} 结构。
- 格式归一、异常兜底、解析质量抽样校验。

Phase 2 · 切分与双通道索引

约 2 周

- 实现语义 / 层级 / Late Chunking，附加章节路径等元数据。
- 向量通道：BGE-M3 嵌入 → 写入 Qdrant，建立 BM25 倒排（或用 Qdrant 稀疏向量）。
- 图谱通道：LightRAG 抽取实体关系与社区摘要，构建图索引。
- 索引版本化，支持增量更新与删除。

Phase 3 · 检索与生成服务

约 2-3 周

- Query Transform 模块（Rewrite / Expansion / HyDE / RAG-Fusion）。
- Hybrid Retriever：BM25 + 向量 + 图谱子图，RRF 融合。

- Reranker 重排 → LLMlingua 压缩 → 组装带引用的 Prompt。
- 接入本地 LLM（Ollama/vLLM），流式输出，返回原文引用定位。
- FastAPI 封装：鉴权、限流、缓存、批处理接口。

Phase 4 · 评估、观测与上线

持续

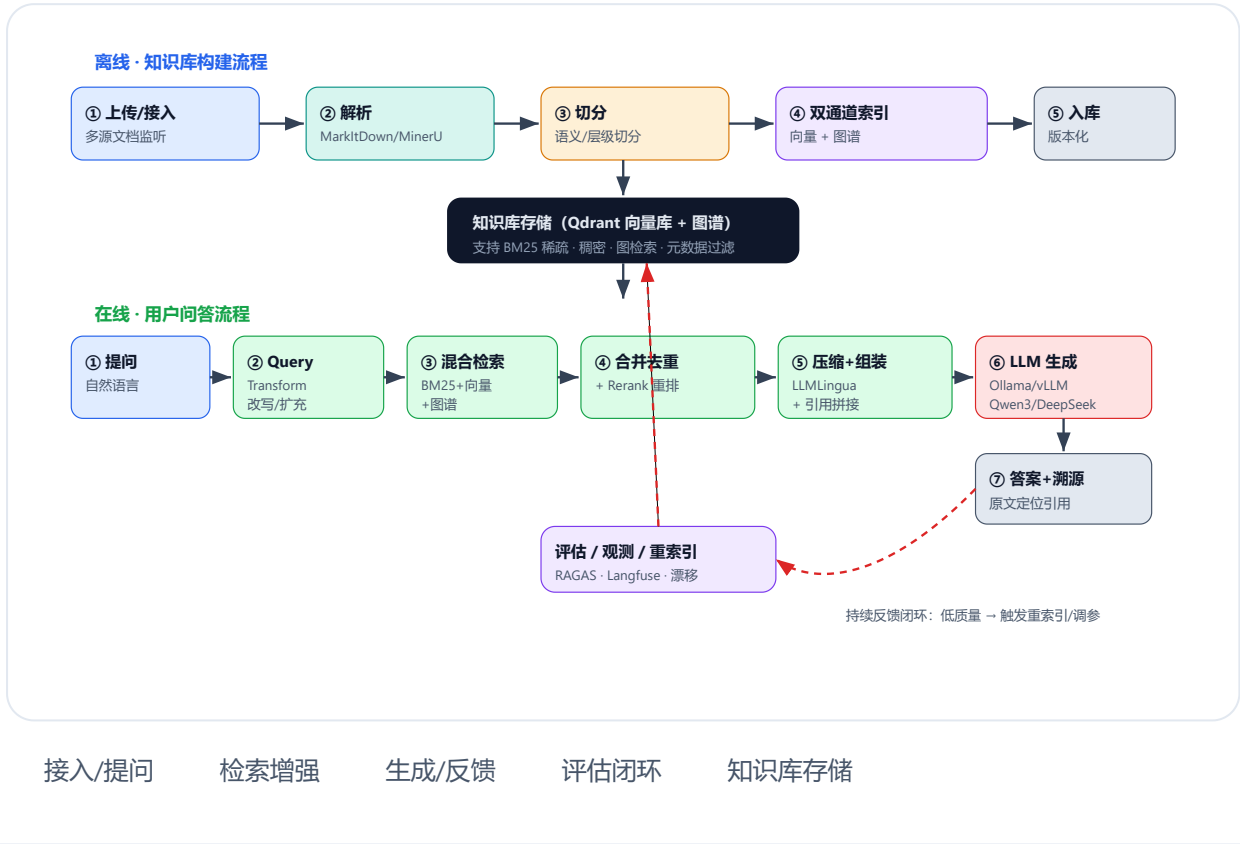
- 构建黄金测试集，用 **RAGAS** 跑忠实度/答案相关性/上下文召回。
- **Langfuse** 观测每一步耗时、token、命中，定位 bad case。
- 前端 UI（Streamlit / Gradio / 自研）供业务试用。
- 监控：索引漂移、查询质量、延迟/成本；触发重索引与模型迭代。

4 系统架构图



离线构建：①②③④ 在线问答：⑤⑥ 数据全程本地私有化，不出域

5 业务流程图



6 技术选型清单（本地优先）

环节	推荐（本地/开源）	备选	说明
文档解析	MarkItDown + MinerU	unstructured, Docling	MinerU 对公式/图表强，MarkItDown 轻量
扫描件 OCR	PaddleOCR / PP-Structure	PaddleOCrv4, RapidOCR	中文识别领先，可本地 GPU 推理
切分	LlamaIndex 语义/层级切分	LangChain Recursive	配合 Late Chunking 保上下文
Embedding	BGE-M3 / Qwen3-Embedding	gte-Qwen2, bge-large-zh	支持稠密+稀疏+多向量，中文优
向量库	Qdrant	Chroma, LanceDB, Milvus, pgvector	支持稀疏向量与过滤，易自托管

环节	推荐（本地/开源）	备选	说明
知识图谱	LightRAG	Microsoft GraphRAG + Neo4j	LightRAG 轻量、复用向量库、上手快
稀疏检索	BM25（rank_bm25 / Qdrant 稀疏）	SPLADE, Elasticsearch	补强专名/编号召回
Reranker	BGE-Reranker-v2-m3 / Qwen3-Reranker	bce-reranker, jina-reranker	Cross-Encoder 重排
Query Transform	LLM 自研 / LlamaIndex 模块	HyDE, RAG-Fusion	改写/扩充/假设文档嵌入
压缩	LLMLingua-2	LongLLMLingua	不影响语义下大幅降 token
生成 LLM	Qwen3 （Ollama） / DeepSeek	ChatGLM4, InternLM3, Phi-4	本地推理，数据不出域
推理服务	Ollama / vLLM	llama.cpp, LMDeploy	vLLM 高并发，Ollama 易用
API 服务	FastAPI	Litestar	流式、鉴权、限流、缓存
评估	RAGAS / DeepEval	TruLens	忠实度/相关性/上下文召回
观测	Langfuse （自托管）	LangSmith(云)	链路追踪、bad case 定位
前端	Streamlit / Gradio	自研 React	快速验证与试用
编排框架	LlamaIndex（推荐 RAG）	LangChain, Haystack	数据连接器与索引丰富

7 关键实现示例（Python 骨架）

以下为最小可运行思路骨架，实际工程请补异常、批处理、并发与监控。

```

# 1) 解析 + 语义切分 (LlamaIndex)
from llama_index.core import SimpleDirectoryReader, SentenceSplitter
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from llama_index.vector_stores.qdrant import QdrantVectorStore

docs = SimpleDirectoryReader("./data", recursive=True).load_data()
splitter = SentenceSplitter(chunk_size=512, chunk_overlap=64)
nodes = splitter.get_nodes_from_documents(docs)

# 2) 向量索引 (BGE-M3 → Qdrant)
embed = HuggingFaceEmbedding(model_name="BAAI/bge-m3")
vs = QdrantVectorStore(client=client, collection_name="kb")
index = VectorStoreIndex(nodes, embed_model=embed, vector_store=vs)

# 3) 图谱索引 (LightRAG)
from lightrag import LightRAG, QueryParam
import asyncio
rag = LightRAG(working_dir="./graph_storage", embedding_func=embed, llm=local_llm)
asyncio.run(rag.ainsert("\n".join(d.text for d in docs)))

# 4) 混合检索 + Rerank + 生成
from llama_index.core.retrievers import QueryFusionRetriever
retriever = QueryFusionRetriever([index.as_retriever(), bm25_retriever],
                                mode="reciprocal_rerank", similarity_top_k=10)
nodes = retriever.retrieve(rewrite_query(q)) # 先 Query Transform
ranked = reranker.compress(query, nodes, top_n=5) # BGE/Qwen3-Reranker
prompt = build_prompt(q, ranked, compress=True) # LLMLingua 压缩
answer = local_llm.stream(prompt, citations=ranked) # 带引用溯源

# 5) 评估 (RAGAS)
from ragas import evaluate; from ragas.metrics import faithfulness, answer_relevancy
score = evaluate(dataset, metrics=[faithfulness, answer_relevancy])

```

落地建议：先用小语料（几十份文档）打通「解析→索引→问答→评估」闭环，用 RAGAS 建立基线；再逐步放大语料、叠加图谱与混合检索，每加一项都用同一测试集对比指标，避免「为了高级而高级」。